02

Foundation Models verstehen

Trainingsdaten · Modell und Größe · Post-Training · Sampling · strukturierte Ausgaben

Cloud-native AI · B.Sc. Angewandte Informatik · HTWG Konstanz · WS 2026/27 · Grundlage: Huyen, AI Engineering, Kap. 2

Lernziele

- ✓ Erklären, wie Trainingsdaten die Stärken und Schwächen eines Modells prägen, besonders für Deutsch und für Fachdomänen.
- ✓ Den Weg einer Anfrage durch ein Sprachmodell beschreiben: Tokens, Transformer, Wahrscheinlichkeiten, Sampling, Prefill und Decode.
- ✓ Aus Parameterzahl und Datentyp den Speicherbedarf eines Modells abschätzen und Mixture-of-Experts-Modelle einordnen.
- ✓ Base-, Instruct- und Chat-Modelle unterscheiden und erklären, was SFT und Präferenz-Finetuning bewirken.
- ✓ Temperature, Top-k und Top-p für eine Anwendung begründet einstellen.
- ✓ Strukturierte Ausgaben erzwingen und validieren, und erklären, warum Modelle inkonsistent sind und halluzinieren.

Worum es geht

In der Übung zu VL 1 habt ihr zwei Modelle gefragt, ob die HTWG Konstanz eine private oder eine staatliche Hochschule ist. Manche Antworten waren richtig, manche falsch, und wer dieselbe Frage zweimal gestellt hat, bekam nicht immer dieselbe Antwort. Für einen Campus-Assistenten, dem Studierende vertrauen sollen, ist das ein Problem.

Um damit umzugehen, müsst ihr kein Modell trainieren können. Ihr müsst aber verstehen, welche Entscheidungen beim Bau eines Modells getroffen wurden und welche Folgen sie für eure Anwendung haben. Huyen nennt vier solche Entscheidungen, und genau in dieser Reihenfolge geht dieses Kapitel vor:

1. **Trainingsdaten:** Woraus hat das Modell gelernt?
2. **Modell und Größe:** Wie ist es aufgebaut, und wie groß ist es?
3. **Post-Training:** Wie wurde aus einem Textvervollständiger ein hilfreicher Assistent?
4. **Sampling:** Wie wird aus Wahrscheinlichkeiten eine konkrete Antwort?

Der letzte Punkt wird am häufigsten übersehen. Huyen nennt Sampling eines der am meisten unterschätzten Konzepte der KI, weil es viele scheinbar rätselhafte Verhaltensweisen erklärt, darunter Inkonsistenz und Halluzinationen. Gleichzeitig ist es der Hebel, den ihr als Entwickelnde direkt in der Hand habt.

★ Ihr müsst kein Modell bauen können, um es gut einzusetzen. Ihr müsst aber wissen, woraus es gelernt hat, wie groß es ist, wie es nachtrainiert wurde und wie es seine Antworten auswählt. Daraus folgen Modellwahl, Kosten und die Einstellungen eures API-Aufrufs.

Trainingsdaten

Ein Modell ist nur so gut wie die Daten, aus denen es gelernt hat. Kommt eine Sprache oder ein Fachgebiet in den Trainingsdaten kaum vor, kann das Modell darin auch wenig.

Die meisten großen Modelle lernen zu einem guten Teil aus **Common Crawl**, einer frei verfügbaren Sammlung von Webseiten. Eine gemeinnützige Organisation durchsucht dafür regelmäßig das Web, 2022 und 2023 waren es etwa zwei bis drei Milliarden Seiten pro Monat. Die Qualität ist entsprechend gemischt: Neben Lexika und Fachseiten stecken darin Clickbait, Falschmeldungen und Propaganda. Modellanbieter filtern die Daten deshalb mit Heuristiken, legen aber immer seltener offen, was genau sie verwenden.

Sprachen

Englisch dominiert das Web. In Common Crawl macht es fast die Hälfte der Daten aus (45,9 %). Deutsch liegt mit 5,9 % auf Platz drei, knapp hinter Russisch. Das klingt wenig, ist aber komfortabel: Viele Sprachen mit Hunderten Millionen Sprechenden kommen nur auf Bruchteile eines Prozents und gelten als **low-resource**.

Daraus folgen zwei Effekte, die ihr auch für Deutsch spüren werdet:

- **Qualität:** Allgemeine Modelle sind in Englisch messbar besser als in anderen Sprachen. GPT-4 löste Mathematikaufgaben auf Englisch mehr als dreimal so oft wie auf Armenisch oder Farsi.
- **Kosten und Latenz:** Modelle rechnen nicht in Wörtern, sondern in **Tokens**, und die Zerlegung in Tokens ist für Englisch optimiert. Für denselben Inhalt braucht Hindi im Median etwa viermal, Birmanisch etwa zehnmal so viele Tokens wie Englisch. Da Anbieter pro Token abrechnen und jedes Token Rechenzeit kostet, ist dieselbe Antwort in diesen Sprachen teurer und langsamer. Auch Deutsch braucht mehr Tokens als Englisch. Wie viele, messt ihr in der Übung selbst.

💡 **Tipp:** Testet ein Modell in der Sprache, in der eure Nutzerinnen und Nutzer schreiben. Ein gutes Ergebnis in einem englischen Benchmark sagt wenig über die Qualität auf Deutsch.

Domänen

Dasselbe gilt für Fachgebiete. Allgemeine Modelle beantworten Alltagsfragen zu fast jedem Thema, weil das Web zu fast jedem Thema etwas enthält. Für Aufgaben, deren Daten nicht öffentlich im Web stehen, sind sie schwach. Huyen nennt Wirkstoffforschung (Protein- und DNA-Daten) und Krebsvorsorge (Röntgen- und MRT-Bilder). Dafür gibt es **domänenspezifische Modelle** wie AlphaFold für Proteinstrukturen.

Für den Campus-Assistenten heißt das: Ein allgemeines Modell kennt allgemeine Hochschulbegriffe, aber nicht die Prüfungsordnung der HTWG, nicht den aktuellen Mensaplan und nicht die Fristen dieses Semesters. Dieses Wissen müssen wir dem Modell zur Laufzeit geben. Wie das geht, ist Thema von VL 4.

> ★ Was nicht in den Trainingsdaten steht, kann das Modell nicht wissen. Für Deutsch ist die Lage gut, aber nicht so gut wie für Englisch. Für lokales Wissen wie Prüfungsordnungen ist die Lage schlecht, egal wie groß das Modell ist.

Modell und Größe

Vom Text zum Token

Ein Sprachmodell sieht keine Buchstaben und keine Wörter, sondern **Tokens**. Ein Token ist ein Stück Text, oft ein Wort, ein Wortteil oder ein Satzzeichen. Ein **Tokenizer** zerlegt den Text, jedes Token bekommt eine Nummer aus dem **Vokabular** des Modells. Llama 3 hat ein Vokabular von 128 000 Tokens.

Häufige englische Wörter sind meist ein einziges Token. Seltene oder lange Wörter werden zerlegt, aus „Mensa“ werden zum Beispiel „␣Mens“ und „a“. Das Zeichen ␣ steht für das Leerzeichen, das zum Token gehört. Als Faustregel entsprechen 100 Tokens im Englischen etwa 75 Wörtern. Im Deutschen sind es weniger Wörter pro Token, weil zusammengesetzte Wörter wie „Prüfungsanmeldung“ in mehrere Tokens zerfallen.

Der Transformer als Blackbox

Fast alle heutigen Sprachmodelle beruhen auf der **Transformer-Architektur** (Vaswani et al., 2017). Ihr Kern ist der **Attention-Mechanismus**: Bei jedem neuen Token kann das Modell auf alle bisherigen Tokens zurückgreifen und gewichten, welche gerade wichtig sind. Huyen vergleicht das mit einer Prüfungsfrage zu einem Buch. Ältere Architekturen mussten mit einer Zusammenfassung des Buchs antworten. Ein Transformer darf jede Seite nachschlagen.

Für die Anwendung genügt es, das Modell als Stapel gleichartiger **Transformer-Blöcke** zu sehen, Llama 3 mit 8 Milliarden Parametern hat 32 davon, die 70-Milliarden-Variante 80. Wichtig sind drei Folgen dieses Aufbaus:



Abbildung 1: Der Weg einer Anfrage durch das Modell. Schritt 3 bis 5 laufen für jedes Ausgabe-Token erneut.

Das Modell erzeugt immer nur ein Token. Am Ende jedes Durchlaufs steht eine Wahrscheinlichkeit für jedes Token im Vokabular. Daraus wird ein Token ausgewählt, an die Eingabe angehängt, und das Ganze beginnt von vorn. Man nennt solche Modelle **autoregressiv**.

Inferenz hat zwei Phasen. Im **Prefill** verarbeitet das Modell alle Eingabe-Tokens parallel. Das geht schnell, dauert bei langen Eingaben aber spürbar und bestimmt die **Time to First Token**, die ihr in VL 1 gemessen habt. Im **Decode** entsteht ein Ausgabe-Token nach dem anderen, streng nacheinander. Das bestimmt die **Time per Output Token**. Lange Antworten sind deshalb teuer und langsam, lange Eingaben vor allem teuer.

Der Kontext ist begrenzt. Für jedes bisherige Token muss das Modell Zwischenergebnisse speichern. Je länger die Folge, desto mehr Speicher braucht es. Deshalb hat jedes Modell eine maximale **Kontextlänge**, bei Llama 2 waren es 4 000 Tokens, bei Llama 3 sind es 128 000. Eingabe und Ausgabe zusammen müssen hineinpassen.

> ⚠ **Achtung:** „Das Modell liest meine Frage und denkt dann über die Antwort nach.“ Es gibt kein Nachdenken vor der Antwort. Jedes Token entsteht in einem einzigen Durchlauf, und das nächste Token baut auf dem vorigen auf. Auch ein schlechter Anfang wird fortgesetzt.

Wie groß ist ein Modell?

Die Zahl hinter dem Modellnamen ist die Zahl der **Parameter**, also der gelernten Gewichte: `llama3.2:3b` hat 3 Milliarden, `llama-3.3-70b-instruct` 70 Milliarden. Innerhalb einer Modellfamilie ist das größere Modell in der Regel besser. Neuere Modelle schlagen aber oft deutlich größere ältere: Llama 3 mit 8 Milliarden Parametern übertrifft Llama 2 mit 70 Milliarden im MMLU-Benchmark.

Für den Betrieb ist die Parameterzahl vor allem eine **Speicherfrage**. Jeder Parameter belegt je nach Datentyp eine feste Zahl von Bytes:

Modell	Parameter	Mindestens nötiger Speicher für die Gewichte (16 Bit, also 2 Byte pro Parameter)
`qwen2.5:0.5b`	0,5 Mrd.	ca. 1 GB, läuft auf jedem Laptop
`llama3.2:3b`	3 Mrd.	ca. 6 GB, mit Quantisierung (4 Bit) etwa 2 GB
`meta-llama-3.1-8b-instruct`	8 Mrd.	ca. 16 GB, eine gute Consumer-GPU
`llama-3.3-70b-instruct`	70 Mrd.	ca. 140 GB, mehrere Rechenzentrums-GPUs

Die Faustregel lautet: **Parameter × Bytes pro Parameter.** Der tatsächliche Bedarf liegt höher, weil zur Laufzeit noch die Zwischenergebnisse für den Kontext dazukommen. Dass Ollama auf euren Laptops trotzdem Modelle mit mehreren Milliarden Parametern ausführt, liegt an der **Quantisierung**: Die Gewichte werden mit 4 statt 16 Bit gespeichert. Das behandeln wir in VL 10.

Mixture of Experts

Eine Parameterzahl kann täuschen. Bei **Mixture-of-Experts-Modellen** (MoE) besteht das Modell aus vielen Gruppen von Parametern, den **Experten**. Für jedes Token arbeiten nur wenige davon. Mixtral 8x7B hat insgesamt 46,7 Milliarden Parameter, pro Token sind aber nur 12,9 Milliarden aktiv. Es rechnet deshalb so schnell und günstig wie ein Modell mit 12,9 Milliarden Parametern, braucht aber Speicher für alle 46,7 Milliarden.

Genau so ist das Modell aufgebaut, das wir in der Academic Cloud nutzen: In `qwen3-30b-a3b-instruct-2507` steht `30b` für rund 30 Milliarden Parameter insgesamt und `a3b` für rund 3 Milliarden **aktive** Parameter pro Token.

Daten, Rechenleistung und Grenzen des Wachstums

Neben der Parameterzahl beschreiben zwei weitere Zahlen die Größe eines Modells: die Zahl der **Trainingstokens** (Llama 3 wurde mit 15 Billionen Tokens trainiert) und die Rechenleistung in **FLOPs**, die das Training gekostet hat. Die **Chinchilla-Skalierungsregel** sagt, dass für ein bestimmtes Rechenbudget etwa 20 Trainingstokens pro Parameter optimal sind. Viele Anbieter trainieren kleine Modelle bewusst länger, weil kleine Modelle im Betrieb billiger sind.

Zwei Engpässe zeichnen sich ab. Die öffentlich verfügbaren Textdaten im Web gehen zur Neige, und das Web füllt sich mit KI-generierten Texten. Außerdem brauchen Rechenzentren immer mehr Strom. Für euch als Entwickelnde heißt das vor allem: Mit kleineren, gut trainierten Modellen kommt man oft weiter als gedacht.

> ★ Die Parameterzahl bestimmt den Speicherbedarf (Parameter × Bytes) und grob die Qualität innerhalb einer Modellfamilie. Bei MoE-Modellen bestimmen die **aktiven** Parameter Geschwindigkeit und Kosten pro Token, die **gesamten** Parameter den Speicherbedarf.

Post-Training

Nach dem Pre-Training kann ein Modell zwar Text fortsetzen, aber noch nicht antworten. Huyen nennt zwei Probleme: Es ist auf das Vervollständigen von Text optimiert statt auf Gespräche, und es hat aus ungefilterten Webdaten auch Unerwünschtes gelernt. Ein Freund von Huyen hat das so beschrieben: Ein vortrainiertes Modell redet wie eine Webseite, nicht wie ein Mensch.

Das **Post-Training** löst beide Probleme in zwei Schritten:

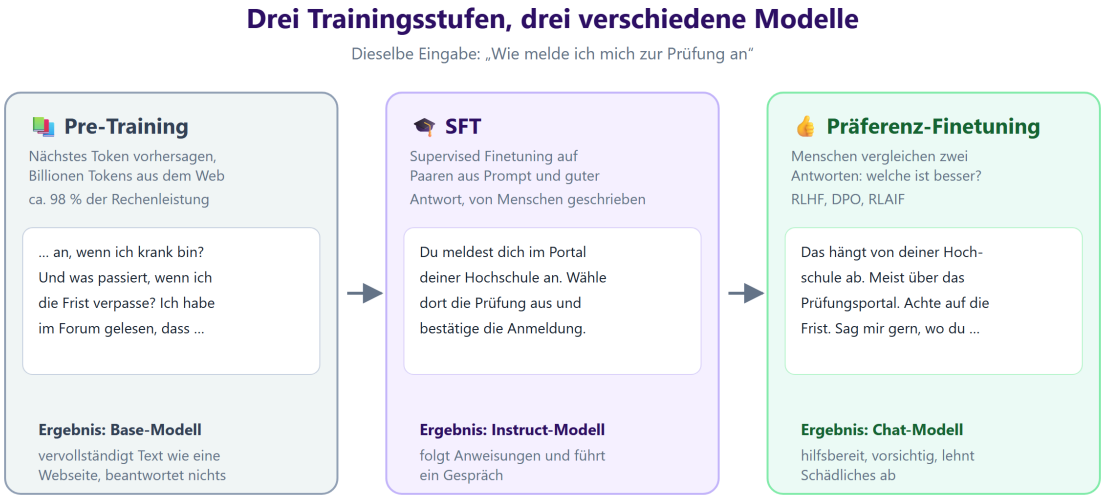


Abbildung 2: Dieselbe Eingabe an drei Stufen desselben Modells.

1. **Supervised Finetuning (SFT):** Das Modell wird mit Paaren aus Prompt und guter Antwort weitertrainiert, die Menschen geschrieben haben. OpenAI nutzte für InstructGPT rund 13 000 solcher Paare. Das Modell lernt dadurch das Format eines Gesprächs.
2. **Präferenz-Finetuning:** Menschen vergleichen jeweils zwei Antworten und wählen die bessere aus. Das ist leichter, als eine perfekte Antwort zu schreiben. Daraus lernt das Modell, Antworten zu geben, die Menschen bevorzugen. Bekannte Verfahren sind **RLHF** (Reinforcement Learning from Human Feedback, mit einem eigenen

Bewertungsmodell), **DPO** (Direct Preference Optimization, einfacher, genutzt für Llama 3) und **RLAIF**, bei dem ein Modell statt Menschen bewertet.

Das Post-Training ist im Verhältnis billig: Bei InstructGPT entfielen etwa 2 % der Rechenleistung darauf, 98 % auf das Pre-Training. Man kann es sich so vorstellen, dass das Pre-Training Wissen aufbaut und das Post-Training lehrt, dieses Wissen hilfreich einzusetzen.

Was das für die Modellwahl heißt

Viele Modelle gibt es in mehreren Varianten. Am Namen erkennt ihr die Stufe:

- **Base** oder ohne Zusatz: nur vortrainiert. Setzt Text fort, beantwortet keine Fragen. Für Anwendungen ungeeignet.
- **Instruct** oder **Chat**: nach SFT und meist auch Präferenz-Finetuning. Das ist die Variante für eure Anwendungen.

Am Modellnamen `qwen3-30b-a3b-instruct-2507` könnt ihr jetzt alles ablesen: Familie Qwen 3, 30 Milliarden Parameter, davon 3 Milliarden aktiv, Instruct-Variante, Stand Juli 2025.

> ⚠ **Achtung:** Präferenz-Finetuning macht ein Modell hilfsbereit, nicht wahrhaftig. Bewertet wird, welche Antwort Menschen lieber lesen. Eine flüssige, selbstbewusste Antwort wird oft bevorzugt, auch wenn sie falsch ist.

Sampling

Am Ende jedes Durchlaufs berechnet das Modell für jedes Token im Vokabular eine Zahl, die **Logits**. Eine **Softmax**-Funktion macht daraus Wahrscheinlichkeiten, die sich zu 1 aufsummieren. **Sampling** ist der Schritt, der aus dieser Verteilung ein konkretes Token auswählt.

Die naheliegende Strategie, immer das wahrscheinlichste Token zu nehmen, heißt **Greedy Decoding**. Für eine Klassifikation ist das sinnvoll. Für Text führt es zu langweiligen, oft repetitiven Antworten. Deshalb ziehen Modelle das nächste Token meist zufällig **gemäß** der Verteilung: Hat „Schnitzel“ 40 % und „Pasta“ 27 %, wird „Schnitzel“ in 40 % der Fälle gewählt. Drei Stellschrauben verändern diese Ziehung.

Temperature

Vor der Softmax werden die Logits durch die **Temperature** geteilt. Huyens Beispiel: Zwei Tokens A und B haben die Logits 1 und 2. Bei Temperature 1 wird B mit 73 % gewählt, bei Temperature 0,5 mit 88 %. Kleine Werte machen die Verteilung also spitzer, große Werte flacher.

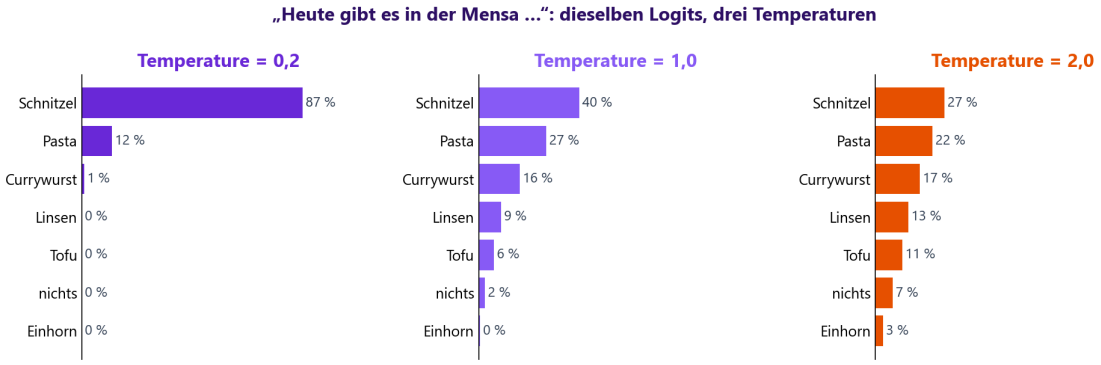


Abbildung 3: Dieselben Logits bei drei Temperaturen. Bei 0,2 gewinnt fast immer „Schnitzel“, bei 2,0 hat sogar „Einhorn“ eine Chance.

- **Niedrige Temperature** (0 bis 0,3): Antworten sind konsistent und vorhersagbar. Richtig für Extraktion, Klassifikation und alles, was ein Programm weiterverarbeitet. Bei Temperature 0 nimmt das Modell schlicht das wahrscheinlichste Token.
- **Mittlere Temperature** (um 0,7): oft empfohlen für kreative Aufgaben, ein Kompromiss aus Vielfalt und Zusammenhang.
- **Hohe Temperature** (über 1): seltene Tokens werden wahrscheinlicher. Kreativ, aber zunehmend unzusammenhängend. Anbieter begrenzen den Wert meist auf 0 bis 2.

Top-k und Top-p

Zwei weitere Strategien schneiden den unwahrscheinlichen Rand der Verteilung ab, bevor gezogen wird:

- **Top-k:** Nur die k wahrscheinlichsten Tokens kommen in Frage, typisch sind Werte zwischen 50 und 500.
- **Top-p** (Nucleus Sampling): Die Tokens werden nach Wahrscheinlichkeit sortiert und so lange aufsummiert, bis die Summe p erreicht. Nur diese Tokens kommen in Frage. Übliche Werte liegen bei 0,9 bis 0,95.

Der Vorteil von Top-p ist, dass sich die Auswahl der Situation anpasst. Bei „Antworte nur mit Ja oder Nein“ bleiben zwei Tokens übrig, bei einer offenen Frage viele. Ein Verwandter ist **Min-p**: Ein Token muss eine Mindestwahrscheinlichkeit erreichen.

Stoppbedingungen und Logprobs

Ein Modell erzeugt Tokens, bis ein **Stopp-Token** kommt oder eine Höchstzahl erreicht ist (`max_tokens`). Eine harte Grenze spart Zeit und Geld, kann aber eine Antwort mitten im Satz abschneiden. Bei JSON fehlt dann die schließende Klammer.

Manche APIs liefern zu jedem Token die **Logprobs**, die logarithmierten Wahrscheinlichkeiten. Daran seht ihr, wie sicher sich das Modell bei einem Token war. Das ist nützlich für Klassifikation und Fehlersuche. Nicht alle Anbieter geben Logprobs heraus, auch weil sie das Nachbauen eines Modells erleichtern.

Test-Time Compute

Statt eine Antwort zu erzeugen, kann man mehrere erzeugen und die beste auswählen. Huyen nennt das **Test-Time Compute**. Ausgewählt wird zum Beispiel:

- nach der höchsten durchschnittlichen Wahrscheinlichkeit der Tokens,
- mit einem Bewertungsmodell (**Verifier**). OpenAI fand, dass ein Verifier etwa so viel bringt wie ein 30-mal größeres Modell,
- per Mehrheitsentscheid bei Aufgaben mit eindeutiger Antwort (**Self-Consistency**). Google ließ Gemini jede MMLU-Frage 32-mal beantworten und nahm die häufigste Antwort,
- mit einer einfachen Regel, etwa „so lange erzeugen, bis die SQL-Abfrage gültig ist“.

Der Preis ist klar: Zwei Antworten kosten etwa doppelt so viel wie eine. Auch sogenannte Reasoning-Modelle, die vor der eigentlichen Antwort lange Gedankengänge erzeugen, investieren mehr Rechenzeit pro Anfrage, um bessere Antworten zu bekommen.

Strukturierte Ausgaben

Eine Anwendung braucht vom Modell oft keinen Fließtext, sondern Daten: ein JSON-Objekt mit festen Feldern, eine gültige SQL-Abfrage oder eine Kategorie aus einer festen Liste. Das gilt besonders, wenn ein Programm die Ausgabe weiterverarbeitet, etwa ein Werkzeug in einem Agenten (VL 5). Huyen ordnet die Techniken nach Stärke:

Technik	Wie sie wirkt
Prompting	„Antworte nur mit JSON nach diesem Schema.“ Funktioniert oft, garantiert aber nichts. Je nach Modell sind zwischen 0 und über 90 % der Ausgaben gültig.
Nachbearbeitung	Typische Fehler per Code reparieren, etwa eine fehlende Klammer. LinkedIn steigerte so den Anteil gültiger YAML-Ausgaben von 90 auf 99,99 %.
Test-Time Compute	So lange neu erzeugen, bis die Ausgabe gültig ist.
Constrained Sampling	Beim Sampling werden alle Tokens gestrichen, die die Grammatik verletzen würden. Nach einer öffnenden Klammer `{` ist zum Beispiel nur ein Feldname erlaubt.
Finetuning	Das Modell auf Beispiele im gewünschten Format nachtrainieren. Am zuverlässigsten, aber aufwendig.

Viele APIs bieten Constrained Sampling heute als Option an. Mit `response_format` übergebt ihr ein JSON-Schema, und das Modell kann nur noch Ausgaben erzeugen, die dazu passen. Ein bloßer **JSON-Modus** garantiert dagegen nur gültiges JSON, nicht die richtigen Felder.

⚠ **Achtung:** Auch eine Ausgabe, die zum Schema passt, kann inhaltlich falsch sein. Das Schema garantiert die Form, nicht die Wahrheit. Und eine zu knappe Grenze bei `max_tokens` schneidet auch strukturierte Ausgaben ab.

Die Wahrscheinlichkeitsnatur von KI

Weil jede Antwort eine Stichprobe aus einer Verteilung ist, verhalten sich Modelle **probabilistisch**. Glaubt ein Modell zu 70 %, dass die vietnamesische Küche die beste der Welt ist, und zu 30 %, dass es die italienische ist, dann antwortet es in 70 % der Fälle „vietnamesisch“. Für kreative Aufgaben ist das ein Gewinn. Für alles, was verlässlich sein muss, ist es eine Herausforderung. Zwei Folgen sind besonders wichtig.

Inkonsistenz

Inkonsistenz zeigt sich in zwei Formen. **Gleiche Eingabe, unterschiedliche Ausgabe:** Dieselbe Frage zweimal gestellt liefert zwei verschiedene Antworten. Dagegen helfen Caching, eine niedrige Temperature und ein fester **Seed** für den Zufallsgenerator. Eine Garantie gibt es trotzdem nicht, weil selbst die Hardware das Ergebnis beeinflussen kann. **Leicht andere Eingabe, sehr andere Ausgabe:** Schon ein anderer Großbuchstabe kann die Antwort ändern. Dagegen helfen nur sorgfältige Prompts (VL 3) und gute Evaluation (VL 8).

Halluzinationen

Eine **Halluzination** ist eine Antwort, die nicht auf Fakten beruht. Für Anwendungen, bei denen es auf Fakten ankommt, ist das fatal. 2023 wurde eine Anwaltskanzlei zu einer Geldstrafe verurteilt, weil sie erfundene Urteile aus ChatGPT bei Gericht eingereicht hatte. Für die Ursache gibt es zwei Hypothesen, die sich ergänzen:

- **Selbsttäuschung:** Das Modell unterscheidet nicht zwischen dem, was ihm gegeben wurde, und dem, was es selbst erzeugt hat. Hat es einmal „Die HTWG ist eine private Hochschule“ geschrieben, behandelt es diesen Satz wie eine Tatsache und baut darauf auf. Man spricht von **Snowballing**.
- **Fehlendes Wissen im Training:** Beim SFT lernt das Modell Antworten nachzuahmen, die Menschen mit ihrem eigenen Wissen geschrieben haben. Fehlt dem Modell dieses Wissen, lernt es, plausibel klingende Antworten zu erfinden.

Was hilft? Ganz verhindern lassen sich Halluzinationen nicht. Kurze Antworten verringern das Risiko, weil weniger Tokens weniger Gelegenheiten zum Erfinden bieten. Prompts wie „Wenn du es nicht weißt, sag es“ helfen etwas. Am wirksamsten ist es, dem Modell die nötigen Fakten mitzugeben, statt sich auf sein Gedächtnis zu verlassen. Genau das tun wir ab VL 4.

★ Jede Antwort eines Modells ist eine Stichprobe. Inkonsistenz entsteht durch den Zufall beim Sampling, Halluzinationen entstehen tiefer, im Training und in der Art, wie das Modell auf seinen eigenen Ausgaben aufbaut. Eine Anwendung muss beides einplanen.

Das Kursbeispiel: der Campus-Assistent

Was heißt das alles für unseren Campus-Assistenten? Ein erster Baustein soll Anfragen von Studierenden einordnen, bevor sie beantwortet werden: Um welches Anliegen geht es, zu welchem Studiengang, gibt es eine Frist? Das Ergebnis verarbeitet ein Programm weiter, es muss also strukturiert und verlässlich sein.

```python
from pydantic import BaseModel
from typing import Literal

class Anfrage(BaseModel):
    anliegen: Literal["pruefung", "mensa",
                      "bibliothek", "sonstiges"]
    studiengang: str | None
    frist: str | None
```

Das Pydantic-Modell beschreibt das gewünschte Ergebnis. `Literal` erlaubt nur die vier Kategorien, `str | None` heißt: Das Feld darf leer bleiben, wenn die Mail nichts dazu sagt. Aus dieser Klasse erzeugt Pydantic ein JSON-Schema, das wir an das Modell schicken:

```python
schema = Anfrage.model_json_schema()
antwort = client.chat.completions.create(
    model=os.environ["LLM_MODEL"],
    temperature=0,
    messages=[{"role": "user", "content": mail}],
    response_format={"type": "json_schema",
        "json_schema": {"name": "anfrage",
                        "schema": schema}})
anfrage = Anfrage.model_validate_json(
    antwort.choices[0].message.content)
```

Drei Entscheidungen aus diesem Kapitel stecken darin:

1. `temperature=0`, weil eine Einordnung reproduzierbar sein soll und keine Kreativität braucht.
2. `response_format` mit Schema, damit das Modell per Constrained Sampling nur passende Ausgaben erzeugen kann.
3. `model_validate_json` prüft das Ergebnis trotzdem noch einmal. Schlägt die Prüfung fehl, wirft Pydantic einen Fehler, und die Anwendung kann es erneut versuchen, statt mit kaputten Daten weiterzuarbeiten.

Das Modell bleibt dabei ein austauschbarer Baustein: Name und Adresse stehen weiter in der Konfiguration.

Die Vorlesung heute und Übung 1

In den letzten 30 Minuten der Vorlesung probiert ihr Sampling selbst aus: im **Sampling-Labor**, einer Webseite, in der ihr Temperature, Top-k und Top-p an einer Beispielverteilung verändert, und in Google AI Studio, wo ihr dieselbe Frage mit verschiedenen Einstellungen stellt.

In der Übung startet ihr **Übung 1**, die erste Praktikumsabgabe. Sie verbindet VL 1 und VL 2 und ist auf etwa eine Woche angelegt: Ihr bringt einen Modellaufruf in Docker zum Laufen, tauscht das Modell nur über die Konfiguration, messt, wie stark Antworten bei verschiedenen Temperaturen streuen, zählt Tokens für deutschen und englischen Text und baut die strukturierte Einordnung von Anfragen aus dem Kursbeispiel.

Typische Missverständnisse

⚠ **Achtung:**

- „Das Modell weiß, was an der HTWG gilt.“ Es weiß, was in seinen Trainingsdaten stand. Lokales und aktuelles Wissen fehlt.
- „Ein Modell mit 70 Milliarden Parametern ist immer besser als eines mit 8.“ Nur innerhalb derselben Generation. Neuere kleine Modelle schlagen oft ältere große.
- „Ein 30B-MoE-Modell ist so langsam wie ein dichtes 30B-Modell.“ Für die Geschwindigkeit zählen die aktiven Parameter, für den Speicher alle.
- „Temperature 0 macht das Modell korrekt.“ Sie macht es nur vorhersagbarer. Eine falsche Antwort kommt dann eben zuverlässig falsch.
- „Mit JSON-Schema ist die Ausgabe richtig.“ Sie hat die richtige Form. Ob der Inhalt stimmt, ist eine andere Frage.

Zusammenfassung

★ Foundation Models unterscheiden sich durch vier Entscheidungen. Die **Trainingsdaten** bestimmen, welche Sprachen und Domänen ein Modell beherrscht, Englisch dominiert, Deutsch ist gut vertreten, lokales Wissen fehlt. Die **Architektur** ist fast immer ein Transformer, der Token für Token in zwei Phasen arbeitet: Prefill bestimmt die Zeit bis zum ersten Token, Decode die Zeit pro weiterem Token. Die **Größe** bestimmt den Speicherbedarf, bei MoE-Modellen sind nur wenige Parameter pro Token aktiv. Das **Post-Training** aus SFT und Präferenz-Finetuning macht aus einem Textvervollständiger ein Instruct-Modell. Beim **Sampling** entscheiden Temperature, Top-k und Top-p, wie zufällig die Auswahl ist. Strukturierte Ausgaben erzwingt man am besten per Schema und validiert sie trotzdem. Weil jede Antwort eine Stichprobe ist, sind Inkonsistenz und Halluzinationen eingebaute Eigenschaften, mit denen eine Anwendung rechnen muss.

Kontrollfragen

1. Ein Kommilitone schlägt vor, alle deutschen Anfragen an den Campus-Assistenten erst ins Englische zu übersetzen, weil das Modell auf Englisch besser ist. Nennt einen Vorteil und zwei Nachteile.
2. Ein Modell mit 14 Milliarden Parametern soll in 16-Bit-Genauigkeit auf einer GPU mit 24 GB laufen. Reicht das? Begründet mit einer Abschätzung.
3. Ihr messt eine lange Time to First Token, aber eine kurze Zeit pro weiterem Token. In welcher Phase liegt das Problem, und was an eurer Anfrage könnte die Ursache sein?
4. Für welche zwei der folgenden Aufgaben würdet ihr Temperature 0 wählen, und warum? Mensa-Gericht erfinden, Anfrage einer Kategorie zuordnen, Werbetext variieren, Frist aus einer Mail extrahieren.
5. Ein Modell liefert mit JSON-Schema zuverlässig gültiges JSON, aber die Frist steht manchmal im falschen Feld. Warum hilft das Schema hier nicht, und was könnt ihr tun?

Quellen

- Chip Huyen: *AI Engineering. Building Applications with Foundation Models.* O'Reilly, 2025, Kapitel 2.
- Roland Huß, Daniele Zonca: *Generative AI on Kubernetes.* O'Reilly, 2026, Introduction, Abschnitt *Understanding LLM Fundamentals.* Frei verfügbar über Red Hat.

- Ashish Vaswani et al.: *Attention Is All You Need.* NeurIPS, 2017.